

Technical Summary:

SPUQ: Perturbation-Based Uncertainty Quantification for Large Language Models

Kai-Yu Lu

1 Research Problem and Motivation

Large Language Models (LLMs) have strong text generation capability but frequently produce confidently incorrect outputs, a phenomenon commonly discussed as hallucination. Reliable deployment requires an uncertainty quantification method that outputs a confidence score when reference answers are unavailable, so that low-confidence responses can be treated with skepticism.

Uncertainty in machine learning is commonly categorized into Aleatoric Uncertainty (data-wise uncertainty) and Epistemic Uncertainty (model-wise uncertainty). In LLMs, aleatoric uncertainty reflects intrinsic variability of natural language with multiple valid outputs for the same input, while epistemic uncertainty reflects lack of knowledge or suboptimal modeling that causes sensitivity to small input changes. Prior LLM uncertainty work largely targets aleatoric uncertainty, leaving epistemic uncertainty inadequately explored. Two practical gaps motivate this study: (i) likelihood-based confidence relies on token probabilities that are not exposed by many closed LLM APIs, and (ii) sampling without perturbation can remain overconfident when the model repeatedly reproduces the same incorrect answer.

The technical goal is uncertainty calibration: confidence scores should align with empirical correctness, reducing overconfidence and improving trustworthiness.

2 Related Work

One-pass uncertainty estimation includes likelihood-based measures such as length-normalized generation likelihood or the reciprocal of perplexity. These approaches require access to token-level probabilities and are therefore restricted to APIs that expose the predicted token distribution.

Sampling-based uncertainty estimation measures disagreement among multiple outputs sampled from the same input. Prior work often assumes an “exact match” criterion among outputs, which is suitable for classification-style tasks but not general text generation. Additionally, sampling without perturbation can fail under confident errors because resampling frequently reproduces the same mistaken output, producing a high confidence score despite low accuracy.

Epistemic uncertainty has been associated with adversarial examples and quantified by injecting perturbations at inference time, including methods such as Monte Carlo Dropout in deep learning. Closed LLM APIs limit access to internal stochasticity, motivating input-side perturbation as a practical alternative.

Complementary reliability literature studies prompting and self-consistency style aggregation over reasoning paths. This study focuses on calibrated confidence estimation rather than directly improving task accuracy.

3 Dataset Construction

No new dataset is constructed. Experiments use existing benchmarks spanning summarization and question answering. Dataset scale, licensing, preprocessing, and train-validation-test splits are not specified in the paper.

Table 1: Datasets used for evaluation and their properties as described in the paper. Missing fields are not specified in the paper.

Dataset	Task Type	Input and Output		Notes
XSUM	Summarization	Input: document;	Output: summary	Accuracy definition for calibration: Not specified in the paper.
StrategyQA	Classification (Yes/No) QA	Input: question;	Output: {Yes, No}	Evaluated by exact match.
BoolQ	Classification (Yes/No) QA	Input: question;	Output: {Yes, No}	Evaluated by exact match.
CoQA	Generation QA	Input: conversational question;	Output: free-form answer	Evaluated by F1.
TruthfulQA	Generation QA	Input: question;	Output: free-form answer	Evaluated by F1.

4 Query Protocol and Task Definitions

The method operates on an original inference configuration consisting of a temperature and a prompt. Temperature controls randomness in sampling. The prompt contains the query text, and for chat-oriented models it includes both a user prompt and a system message.

The task is to compute a confidence score $c \in [0, 1]$ for each query, where higher c denotes lower uncertainty. Success is defined by improved calibration, assessed primarily by Expected Calibration Error (ECE) and secondarily by the Pearson correlation ρ between confidence score and accuracy.

Two evaluation settings are defined:

- **Classification-type datasets:** correctness is computed by exact match between the model output and the reference answer.
- **Generation-type datasets:** correctness is computed by the F1 criterion following standard evaluation practice in the referenced datasets.

Hyperparameter selection is performed on a development set. For each tuning run, the development set contains 30 randomly selected samples, and results are averaged over five tuning runs to evaluate robustness to the development set.

5 Modeling Approach

Sampling with Perturbation for Uncertainty Quantification (SPUQ) is a framework that combines (i) sampling to capture aleatoric uncertainty and (ii) input perturbation to expose epistemic uncertainty. The method

contains two modules: a Perturbation Module that generates perturbed inputs and an Aggregation Module that maps multiple sampled outputs to a single confidence score.

5.1 End-to-End Pipeline

Given original temperature T_0 and original prompt x_0 , SPUQ performs:

1. Generate k perturbed input configurations (T_i, x_i) for $i = 1, \dots, k$.
2. Query the LLM with (T_j, x_j) for $j = 0, \dots, k$ where (T_0, x_0) is included, producing outputs $\{y_j\}_{j=0}^k$.
3. Aggregate $\{y_j\}_{j=0}^k$ into a confidence score $c \in [0, 1]$.

5.2 Perturbation Module

The perturbation module is designed for closed LLM APIs and injects perturbations through the model inputs:

- **Temperature perturbation:** either set a temperature offset relative to T_0 across all inputs, or sample temperatures from a specified range. The study reports sampling temperatures uniformly from $[0.0, 2.0]$ for GPT-series APIs and from $[0.0, 1.0]$ for PaLM-series APIs due to API constraints.
- **Prompt perturbation:** lexical variations that preserve meaning using one of the following:
 - **Paraphrasing:** generate k paraphrases of x_0 using an external LLM call, producing $\{x_i\}_{i=1}^k$.
 - **Dummy tokens:** append or prepend low-impact tokens such as newline characters, tab spaces, ellipses, or extra punctuation, yielding $x_i = x_0 + d_i$ or $x_i = d_i + x_0$.
 - **System message perturbation:** for chat-mode models, replace the system message with one randomly selected from a predefined set of semantically similar messages.

5.3 Aggregation Module

The aggregation module generalizes sampling-based confidence estimation beyond exact match by introducing textual similarity. Two categories are evaluated.

5.3.1 Inter-sample aggregation

$$c_{\text{inter}} = \frac{\sum_{i=0, i \neq j}^k s(y_j, y_i) w_i}{\sum_{i=0, i \neq j}^k w_i} \quad (1)$$

Purpose. Compute a confidence score from agreement between multiple outputs, enabling uncertainty estimation when exact match among outputs is insufficient for general text generation.

Symbols. k is the number of perturbed inputs. y_i is the output from the LLM under input configuration (T_i, x_i) , with $i = 0$ denoting the original configuration. j is the index of the output being assessed. $s(y_j, y_i)$ is a textual similarity function between outputs. w_i is a nonnegative weight for sample i .

Intuition. Confidence increases when the selected output y_j is similar to many other sampled outputs. Weighting allows discounting extreme perturbations by emphasizing outputs generated from inputs closer to the original prompt.

Usage. For calibration relative to the original query, j is set to 0. For perturbed prompts, the weight is set as $w_i = s(x_0, x_i)$ to prioritize mild perturbations. Similarity metrics evaluated for $s(\cdot, \cdot)$ include RougeL, BERTScore, and cosine similarity from SentenceBERT embeddings.

5.3.2 Intra-sample aggregation

$$c_{\text{intra}} = \frac{\sum_{i=0}^k c(x_i, y_i)}{k + 1} \quad (2)$$

Purpose. Aggregate per-sample uncertainty estimates into a single confidence score, supporting both APIs that expose token probabilities and APIs that do not.

Symbols. $c(x_i, y_i)$ is a per-sample confidence derived from the i -th input-output pair. k is the number of perturbed inputs. x_i and y_i are the prompt and output for the i -th configuration.

Intuition. Instead of measuring agreement among outputs, confidence is computed for each output independently, then averaged. This enables incorporation of single-sample confidence signals such as likelihood or verbalized confidence.

Usage. Two strategies for $c(x_i, y_i)$ are evaluated: (i) likelihood-based confidence using token probability or reciprocal perplexity when available, and (ii) verbalized uncertainty, where the LLM is prompted after generation to report confidence either as words or a numeric score. For word-based verbalization, the mapping is $c = 0.25$ for “low”, $c = 0.5$ for “medium”, and $c = 0.75$ for “high”.

5.4 Minimal Walkthrough Example

A binary question from StrategyQA illustrates the mechanism. Under the original prompt, GPT-3 outputs “No” with high likelihood (92%) and the answer is incorrect. Under paraphrased prompts, outputs flip between “Yes” and “No” with high likelihood values, revealing prediction instability under meaning-preserving perturbations. Using inter-sample aggregation with exact match as the similarity function for the binary outputs and uniform weights, the confidence relative to the original output becomes $c = 0.50$ because only half of the sampled outputs match the original prediction. This example operationalizes epistemic uncertainty as sensitivity to small perturbations.

Table 2: Minimal SPUQ example in a Yes/No setting based on the motivating StrategyQA case study.

Prompt Variant	Output	Likelihood
Original prompt	No (Incorrect)	92%
Paraphrase 1	Yes	83%
Paraphrase 2	No	97%
Paraphrase 3	Yes	94%
Paraphrase 4	No	95%

6 Empirical Results

6.1 Experimental Setup Summary

LLMs. Five models are evaluated: GPT-3 (text-davinci-003), ChatGPT (gpt-35-turbo-v0301), GPT-4 (gpt-4), PaLM2 (text-bison), and PaLM2-Chat (chat-bison). GPT-series models are accessed via Azure OpenAI API and PaLM-series models via Google Cloud Platform API. Training is not performed because the study evaluates inference-time uncertainty methods.

Baselines. Baselines include length-normalized likelihood (applicable only to GPT-3), verbalized uncertainty, and sampling without perturbation. For generation-type datasets, the sampling baseline replaces exact match with textual similarity for fair comparison, with the similarity metric tuned on a development set.

Metrics. Calibration quality is evaluated by ECE and Pearson correlation ρ between confidence score and accuracy. Accuracy is computed by exact match for classification-type datasets and by F1 for generation-type datasets. The paper describes ECE as the mean absolute difference between each confidence bucket’s accuracy and average confidence score.

6.2 Main Calibration Improvements

Across models and question answering datasets, SPUQ achieves the lowest ECE in most cases and reduces ECE by 50% on average, with improvements ranging from 30% to 70% depending on the dataset and model. This indicates substantially better alignment between confidence scores and empirical correctness, addressing systematic overconfidence.

On the XSUM summarization task, the paper reports the following ECE values:

Table 3: ECE on XSUM summarization. Lower is better calibration. “Not specified” indicates the baseline is not applicable due to API constraints.

LLM	Likelihood	Sampling (no perturbation)	SPUQ
GPT-3	0.386	0.393	0.214
ChatGPT	Not specified in the paper	0.406	0.209

For GPT-3 on XSUM, SPUQ reduces ECE from 0.393 (sampling) to 0.214, a relative reduction of approximately 45.5%. For ChatGPT on XSUM, SPUQ reduces ECE from 0.406 to 0.209, a relative reduction of approximately 48.5%. These reductions demonstrate that perturbation-driven aggregation yields confidence scores that are less overconfident and more predictive of correctness than sampling without perturbation.

6.3 Attribution and Ablation-Style Findings

The improvement over sampling without perturbation is attributed primarily to the perturbation module designed to expose epistemic uncertainty. Both temperature perturbation and prompt perturbation contribute meaningfully to the calibration gains. The motivating case study shows that perturbations can transform a single confidently wrong prediction into a set of unstable predictions, producing a lower confidence score that mitigates overconfidence.

6.4 Hyperparameter Dependence and Robustness

Number of perturbed samples. Calibration improves as the number of perturbed samples k increases and plateaus at approximately $k = 5$, indicating diminishing returns beyond a small number of perturbations.

Temperature perturbation. Increasing temperature during sampling tends to decrease ECE, improving calibration. Random temperature perturbation does not show the same improvement, indicating directional increase is more effective in the reported experiments.

Prompt perturbation. Among the evaluated methods, paraphrasing achieves superior calibration in more than half of the test cases compared to dummy tokens and system message perturbation.

Aggregation choices. RougeL consistently outperforms BERTScore and SentenceBERT cosine similarity as the textual similarity metric for inter-sample aggregation in the reported comparisons. Intra-sample aggregation is also competitive, and no single aggregation method dominates across all test scenarios, motivating evaluation of both inter-sample and intra-sample options.

Robustness to development set choice. Hyperparameter tuning is repeated five times using different development sets. Test outcomes are robust, with ECE standard deviation roughly 10% of the mean. In an example tuning study on TruthfulQA with GPT-4 and base temperature $T_0 = 0.7$, a configuration using paraphrasing with inter-sample aggregation and RougeL achieves ECE 0.082 and correlation 0.690, while alternative tuned configurations yield similar performance.

Table 4: Implementation and hyperparameter details explicitly described in the paper.

Category	Details
Perturbation types	Temperature perturbation; prompt perturbation via paraphrasing, dummy tokens, and system message perturbation (chat-mode).
Temperature ranges	GPT-series: uniform sampling in $[0.0, 2.0]$; PaLM-series: uniform sampling in $[0.0, 1.0]$ (API constraint).
Number of perturbations	k perturbed inputs plus the original input; calibration plateaus around $k = 5$.
Similarity metrics	RougeL, BERTScore, SentenceBERT cosine similarity (for inter-sample aggregation).
Verbalized confidence prompts	“Your confidence is? (low, medium, high)” and “Your confidence is? (a score between 0.0 to 1.0)” in chat or text variants.
Word-to-score mapping	low $\rightarrow$ 0.25, medium $\rightarrow$ 0.5, high $\rightarrow$ 0.75.
Development tuning	30 randomly selected samples per development set; five tuning runs reported for SPUQ.
Training setup	Not specified in the paper (methods are inference-time UQ procedures).
Compute and cost	Additional cost from multiple generations and paraphrasing; parallelization yields stated $O(1)$ complexity for latency.

6.5 Advantages and Disadvantages Observed

Advantages. SPUQ improves calibration substantially over likelihood-only and sampling-only baselines, particularly in settings where the model is confidently wrong, by converting perturbation-induced instability into lower confidence. The aggregation module extends sampling-based uncertainty estimation beyond exact match, improving applicability to general text generation.

Disadvantages. The approach incurs additional computational cost due to multiple generations and paraphrasing, increasing latency. The paper also notes that the current implementation and analysis for system message perturbation are constrained to basic system messages.

7 Summary

SPUQ is an inference-time uncertainty quantification framework for LLMs that targets both aleatoric uncertainty through sampling and epistemic uncertainty through input perturbation. The method generates perturbed inputs by modifying temperature and prompts, samples outputs for each variant, and aggregates the outputs into a calibrated confidence score using either inter-sample similarity-based agreement or intra-sample per-output confidence signals. Empirical evaluation across five LLMs and multiple datasets reports a 50% average reduction in Expected Calibration Error, with improvements ranging from 30% to 70%, and demonstrates that the perturbation module is the primary driver of gains over sampling without perturbation.

Limitations and future directions (as stated in the paper). Experiments focus on datasets where accuracy can be assessed relatively easily with reference answers. Future work is encouraged on tasks where

accuracy is less well defined, such as conversation and content generation, and on more complex prompt structures. The method adds computational cost due to multiple generations and paraphrasing, though the steps can be parallelized to achieve stated $O(1)$ latency complexity.